

AIA - Arquitetura, Fluxos e Fundamentação Teórica

Assistente documental RAG para consulta ao corpus doutrinário e normativo da Aliança

React + FastAPI + Python

ChromaDB + BM25 + Reranker

OpenAI + LangSmith opcional

1. Pipeline documental

Fluxo offline/build: transforma documentos controlados em artefatos recuperáveis e auditáveis.

- 1 Documentos controlados**
- PDFs e textos doutrinários/normativos
 - Manifestos do corpus e escopo da Aliança

- 2 Extração e normalização**
- PyMuPDF extrai texto bruto
 - Normalização própria reduz ruído sem alterar conteúdo

- 3 Chunking estrutural**
- Capítulos, seções, artigos, perguntas e respostas
 - Preserva unidade doutrinária e contexto local

- 4 Metadados e auditoria**
- document_id, source_category, páginas e namespace
 - Rastreabilidade, filtros e validação do corpus

- 5 Embeddings OpenAI**
- Vetores dos chunks elegíveis
 - Alinhamento entre chunks, manifesto e dimensões

- 6 Índice ChromaDB e JSONL**
- Collection aia_alliance_v1
 - Chunks e vetores persistidos no corpus processado

- 7 Artefato de runtime**
- aia-runtime-corpus.tar.gz
 - Docker reconstrói a imagem sem disco externo

2. Fluxo de resposta

Fluxo online/runtime: recebe a pergunta, recupera evidências e gera resposta citada ou recusa.

- Usuário**
- Pergunta em linguagem natural
 - Pode aplicar filtro por documento

- React + Vite**
- Chat web
 - Exibe resposta, fontes e sugestões

- FastAPI + Uvicorn**
- Endpoint /api/chat
 - Contrato JSON serializável

- RagGenerator**
- Orquestra retrieval, política de evidência, prompt, chamada OpenAI e formatação final.
 - A API reaproveita a instância com functools.lru_cache(maxsize=1).

RetrievalPipeline
Busca híbrida, reordenação neural, recuperação hierárquica e consolidação do contexto.

- Embedding da pergunta**
- OpenAI gera vetor da consulta com o mesmo espaço dos chunks indexados.

- VectorRetriever + ChromaDB**
- Dense retrieval por similaridade vetorial.
 - Retorna chunks com metadados e distância.

- BM25Retriever**
- Busca lexical com rank-bm25.
 - Preserva termos técnicos exatos.

- HybridRetriever**
- Une candidatos vetoriais e lexicais.

- RRF**
- Reciprocal Rank Fusion equilibra rankings.

- CrossEncoder**
- Reranking pergunta + chunk.
 - Modelo sentence-transformers.

- HierarchicalRetriever**
- Usa o chunk recuperado como âncora.
 - Expande para pai/seção documental.

- ContextConsolidator**
- Deduplica, limita tamanho e mantém fontes.
 - Retry por escopo doutrinário/normativo quando necessário.

- EvidencePolicy**
- Decide se o contexto sustenta a pergunta.
 - Evidência insuficiente gera recusa sem chamar o modelo de chat.

- PromptBuilder + OpenAI**
- Monta prompt com contextos e citações.
 - Chat model gera resposta; formatter compacta fontes usadas.

- Resposta final**
- JSON da API contém status, resposta, citações, documentos usados e metadados.
 - A interface apresenta resposta citada, fontes e sugestões de continuidade.

3. Base teórica

Conceitos que fundamentam as decisões técnicas do sistema e a metodologia de avaliação.

- RAG**
- Resposta gerada a partir de evidências recuperadas, não apenas do conhecimento paramétrico.

- Chunking estrutural**
- Divisão respeita capítulos, seções, artigos, perguntas e respostas.

- Dense retrieval**
- Embeddings aproximam perguntas e trechos semanticamente relacionados.

- BM25**
- Busca lexical protege termos doutrinários exatos e expressões técnicas.

- Busca híbrida + RRF**
- Combina sinais semântico e lexical sem depender de um único ranking.

- Cross-Encoder**
- Reavalia pares pergunta + chunk para melhorar precisão dos candidatos.

- Filtros por metadados**
- Controlam corpus, documento, tradição, categoria e namespace.

- Parent retrieval**
- Busca em chunks menores e envia contexto documental mais amplo ao LLM.

- Recusa por evidência**
- Limita respostas quando o corpus recuperado não sustenta a pergunta.

- Avaliação RAG**
- RAGAS/ARES inspiram fidelidade, relevância, contexto e recuperação correta.

- Métricas específicas do domínio**
- Fidelidade documental, acurácia das citações e resposta sem evidência.
 - Taxa de mistura teológica, recusa correta, qualidade dos chunks e separação entre corpus.

- Controle metodológico**
- Corpus controlado pela pesquisadora; upload livre não faz parte do escopo inicial.
 - Avaliação pode usar conjuntos confessionais externos em cenários controlados.

4. Operação, implantação e rastreabilidade

- Docker**
- Imagem única com backend, frontend e corpus.
 - Build instala dependências, baixa reranker e expõe :8000.

- LangSmith opcional**
- Traces: RAG answer, retrieval, policy e prompt.
 - Ajuda a auditar custo, latência e evidência usada.

- Testes e specs**
- pytest cobre retrieval, reranker, pipeline e geração.
 - specs/reports documentam evolução por etapa.

- Segurança e versionamento**
- .env fica fora do Git; corpus runtime é versionado.
 - Chaves são lidas do ambiente e sanitizadas em erros.